

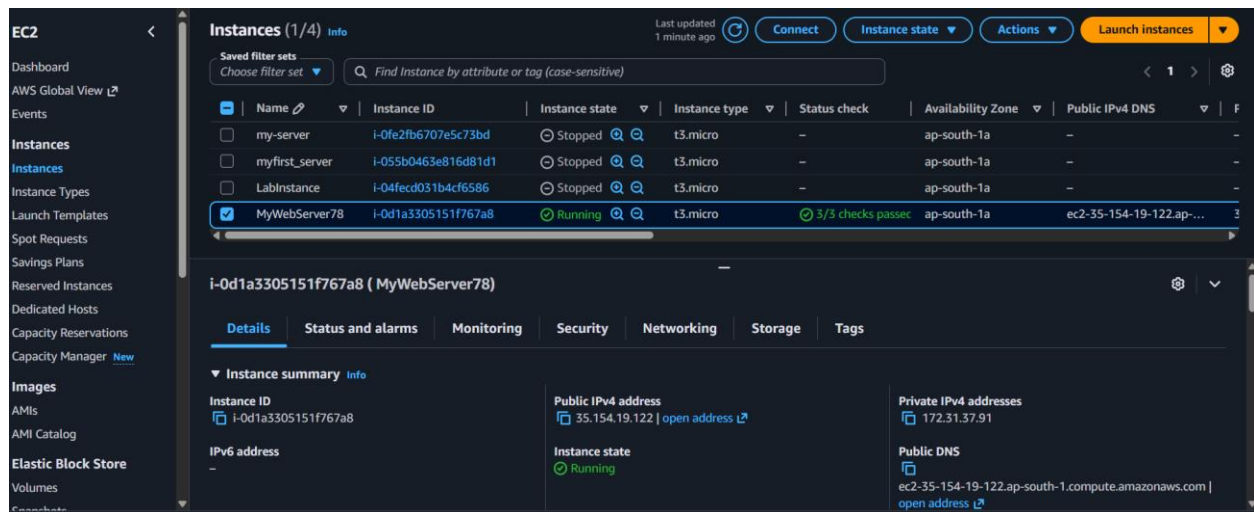
Project

EC2-based Web Server

In this project, we will deploy a web server using Amazon EC2. You will launch an EC2 instance, install either Apache or Nginx, and configure it to serve a basic website.

Step 1: Launch an EC2 Instance

1. Go to the AWS Management Console → EC2.
2. Click Launch Instance.
3. Configure the instance:
 - Name: MyWebServer
 - AMI: Choose Amazon Linux 2 or Ubuntu
 - Instance Type: t2.micro (Free Tier eligible)
 - Key Pair: Create a new key pair or use an existing one.
 - Security Group: Allow SSH (port 22) and HTTP (port 80).
4. Click Launch.



Step 2: Connect to the EC2 Instance

Open a terminal and run:

```
ssh -i your-key.pem ec2-user@your-instance-public-ip
```

Step 3: Install Apache

For Apache (httpd)

Update the package repository: `sudo yum update -y` # Amazon Linux

Install Apache:

`sudo yum install httpd -y` # Amazon Linux

Start and enable Apache:

`sudo systemctl start httpd` # Amazon Linux

`sudo systemctl enable httpd` # Amazon Linux

Verify installation:

`sudo systemctl status httpd` # Amazon Linux

```
ast metadata expiration check: 0:00:26 ago on Sun Jun 21 06:44:31 2026.
Package httpd-2.4.67-1.amzn2023.0.1.x86_64 is already installed.
Dependencies resolved.
Nothing to do.
Complete!
ec2-user@ip-172-31-37-91 ~]$ sudo systemctl start httpd
ec2-user@ip-172-31-37-91 ~]$ sudo systemctl enable httpd
ec2-user@ip-172-31-37-91 ~]$ sudo systemctl status httpd
httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
   Active: active (running) since Wed 2026-06-17 08:18:05 UTC; 3 days ago
     Docs: man:httpd.service(8)
   Main PID: 63637 (httpd)
  Status: "Total requests: 541; Idle/Busy workers 100/0; Requests/sec: 0.00159; Bytes served/sec: 1 B/sec"
    Tasks: 177 (limit: 1014)
   Memory: 15.3M
      CPU: 5min 19.181s
   CGroup: /system.slice/httpd.service
           └─ 63637 /usr/sbin/httpd -DFOREGROUND
           └─ 292343 /usr/sbin/httpd -DFOREGROUND
           └─ 292344 /usr/sbin/httpd -DFOREGROUND
           └─ 292346 /usr/sbin/httpd -DFOREGROUND
           └─ 292372 /usr/sbin/httpd -DFOREGROUND

un 17 08:18:05 ip-172-31-37-91.ap-south-1.compute.internal systemd[1]: Starting httpd.service - The Apache HTTP Server...
un 17 08:18:05 ip-172-31-37-91.ap-south-1.compute.internal httpd[63637]: Server configured, listening on: port 80
un 17 08:18:05 ip-172-31-37-91.ap-south-1.compute.internal systemd[1]: Started httpd.service - The Apache HTTP Server.
un 21 00:00:09 ip-172-31-37-91.ap-south-1.compute.internal systemd[1]: Reloading httpd.service - The Apache HTTP Server...
un 21 00:00:09 ip-172-31-37-91.ap-south-1.compute.internal systemd[1]: Reloaded httpd.service - The Apache HTTP Server.
un 21 00:00:09 ip-172-31-37-91.ap-south-1.compute.internal httpd[63637]: Server configured, listening on: port 80
ec2-user@ip-172-31-37-91 ~]$
```

i-0d1a3305151f767a8 (MyWebServer78)
PublicIPs: 35.154.19.122 PrivateIPs: 172.31.37.91

Step 4: Deploy a Web Page

Edit the default index file:

`sudo echo "<h1>Welcome to My Web Server</h1>" > /var/www/html/index.html`

```
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
   Active: active (running) since Wed 2026-06-17 08:18:05 UTC; 3 days ago
     Docs: man:httpd.service(8)
    Main PID: 63637 (httpd)
   Status: "Total requests: 542; Idle/Busy workers 100/0; Requests/sec: 0.00159; Bytes served/sec: 1 B/sec"
     Tasks: 177 (limit: 1014)
    Memory: 15.3M
      CPU: 5min 19.546s
    CGroup: /system.slice/httpd.service
            └─ 63637 /usr/sbin/httpd -DFOREGROUND
               └─ 292343 /usr/sbin/httpd -DFOREGROUND
                  └─ 292344 /usr/sbin/httpd -DFOREGROUND
                     └─ 292346 /usr/sbin/httpd -DFOREGROUND
                        └─ 292372 /usr/sbin/httpd -DFOREGROUND

Jun 17 08:18:05 ip-172-31-37-91.ap-south-1.compute.internal systemd[1]: Starting httpd.service - The Apache HTTP Server...
Jun 17 08:18:05 ip-172-31-37-91.ap-south-1.compute.internal httpd[63637]: Server configured, listening on: port 80
Jun 17 08:18:05 ip-172-31-37-91.ap-south-1.compute.internal systemd[1]: Started httpd.service - The Apache HTTP Server.
Jun 21 00:00:09 ip-172-31-37-91.ap-south-1.compute.internal systemd[1]: Reloading httpd.service - The Apache HTTP Server...
Jun 21 00:00:09 ip-172-31-37-91.ap-south-1.compute.internal httpd[63637]: Reloading httpd.service - The Apache HTTP Server.
Jun 21 00:00:09 ip-172-31-37-91.ap-south-1.compute.internal httpd[63637]: Server configured, listening on: port 80
[ec2-user@ip-172-31-37-91 ~]$ sudo ss -tulnp | grep :80
tcp        LISTEN 0      511          *:80          *:.*      users:((("httpd",pid=292372,fd=4), ("httpd",pid=292346,fd=4), ("httpd",pid=292344,fd=4), ("httpd",pid=63637,fd=4))
[ec2-user@ip-172-31-37-91 ~]$ sudo echo "<h1>Welcome to My Web Server</h1>" > /var/www/html/index.html
-bash: /var/www/html/index.html: Permission denied
[ec2-user@ip-172-31-37-91 ~]$ echo "<h1>Welcome to My Web Server</h1>" | sudo tee /var/www/html/index.html
<h1>Welcome to My Web Server</h1>
[ec2-user@ip-172-31-37-91 ~]$
```

i-Od1a3305151f767a8 (MyWebServer78)
PublicIPs: 35.154.19.122 PrivateIPs: 172.31.37.91

Step 5: Access the Web Server

Open a browser and go to:

<http://your-instance-public-ip>

<http://35.154.19.122>



Step 6: Configure Auto Start (Optional)

Ensure the web server starts on reboot:

`sudo systemctl enable httpd`

```
tcpd ,pid=63637,fd=4))
[ec2-user@ip-172-31-37-91 ~]$ sudo echo "<h1>Welcome to My Web Server</h1>" > /var/www/html/index.html
-bash: /var/www/html/index.html: Permission denied
[ec2-user@ip-172-31-37-91 ~]$ echo "<h1>Welcome to My Web Server</h1>" | sudo tee /var/www/html/index.html
<h1>Welcome to My Web Server</h1>
[ec2-user@ip-172-31-37-91 ~]$ sudo systemctl enable httpd
[ec2-user@ip-172-31-37-91 ~]$
[ec2-user@ip-172-31-37-91 ~]$
```

i-Od1a3305151f767a8 (MyWebServer78)
PublicIPs: 35.154.19.122 PrivateIPs: 172.31.37.91

CloudShell Feedback Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Conclusion-

We have successfully deployed an EC2-based web server running Apache or Nginx. This setup is commonly used for hosting websites, web applications, or acting as a reverse proxy.

Serverless & Modern Web Hosting

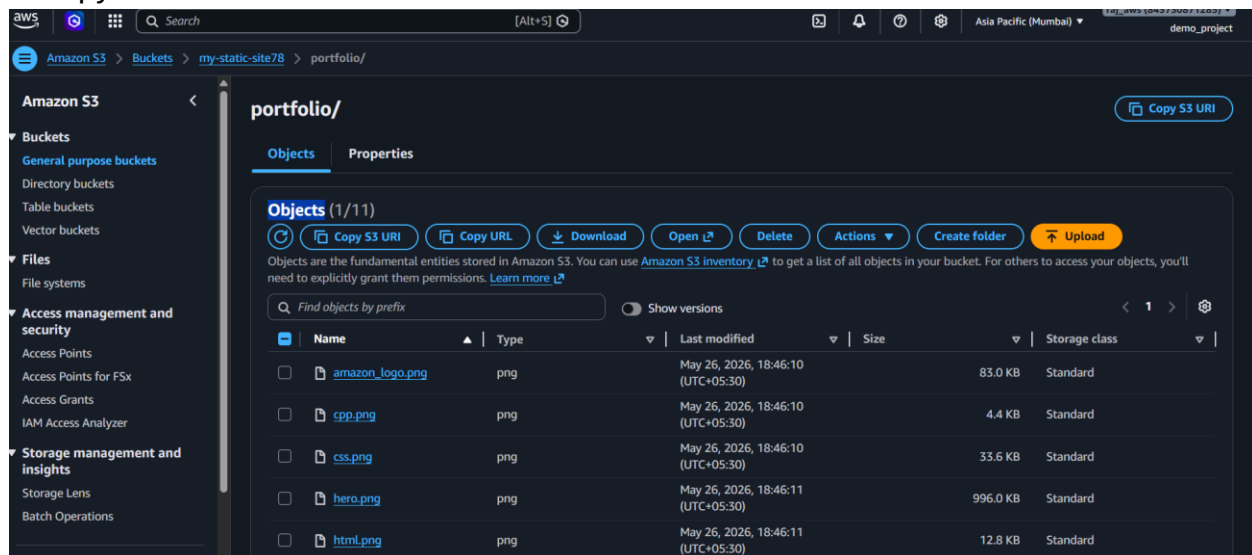
Project 1: Serverless Static Website – Use S3, CloudFront, and Lambda@Edge for global performance.

A serverless static website on AWS eliminates the need for traditional web servers by using Amazon S3 for hosting static content, CloudFront for global content delivery, and Lambda@Edge for dynamic content processing. This setup provides high availability, low latency, and cost efficiency.

Steps to Deploy a Serverless Static Website on AWS

Step 1: Create an S3 Bucket for Website Hosting

1. Open the AWS Management Console → Navigate to S3.
2. Click Create bucket, enter a unique name, and choose the region.
3. Uncheck "Block all public access" (if you want a public website).
4. Upload your static website files (HTML, CSS, JS, images).
5. Enable Static website hosting under Properties:
 - Choose "Enable".
 - Set index.html as the default document.
 - Copy the Bucket Website URL.



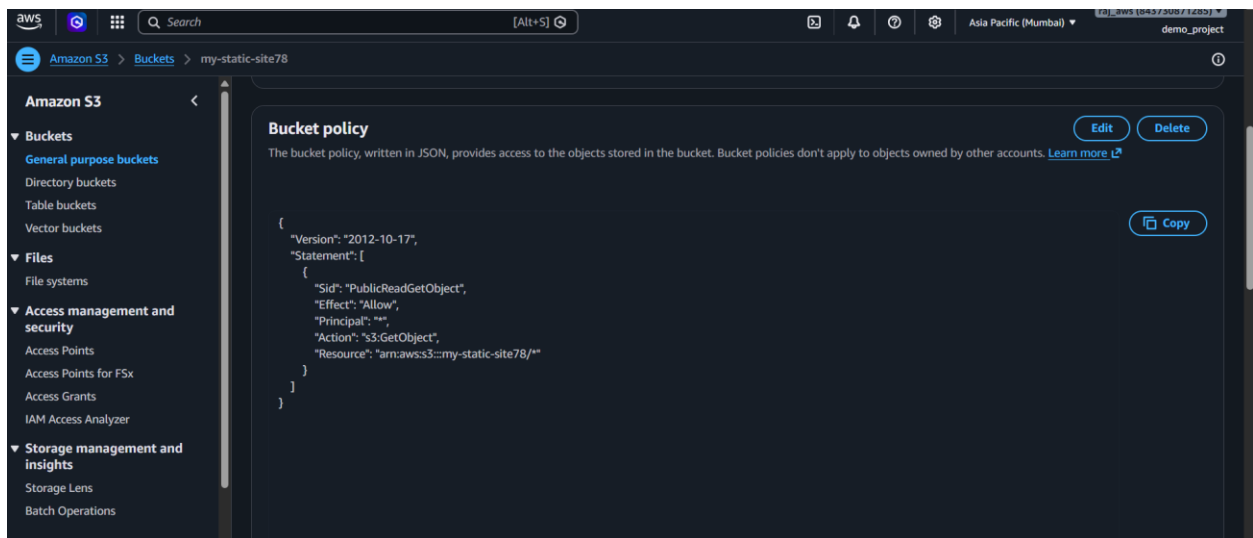
Step 2: Configure Bucket Policy for Public Access

1. Go to Permissions → Click Bucket Policy

Add the following policy to make the website publicly accessible:

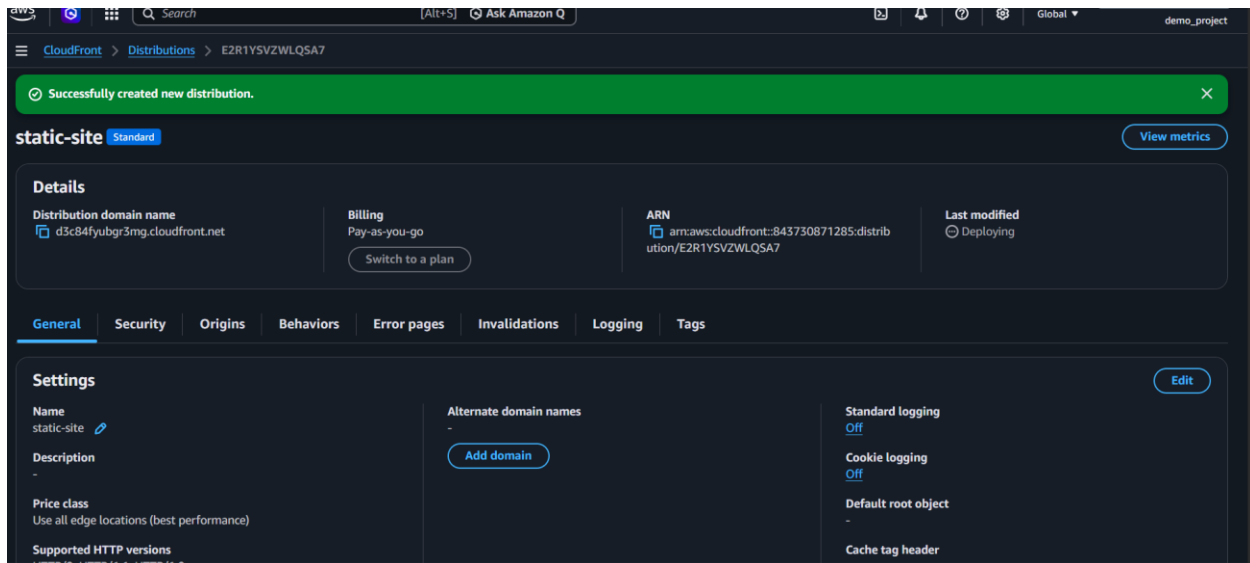
```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::my-static-website/*"
    }
  ]
}
```

2. Save the policy.



Step 3: Create a CloudFront Distribution

1. Go to CloudFront in AWS Console.
2. Click Create Distribution.
3. Under Origin, select your S3 bucket (not the website endpoint, but the bucket itself).
4. Enable "Redirect HTTP to HTTPS".
5. Set Caching Policy to CachingOptimized (for better performance).
6. Click Create and note the CloudFront URL



Conclusion

With this setup, your static website is hosted on S3, served globally using CloudFront, and can be enhanced with Lambda@Edge for additional processing. This architecture ensures scalability, security, and cost efficiency.

